$(y-\hat{y})^2$

minimise ??

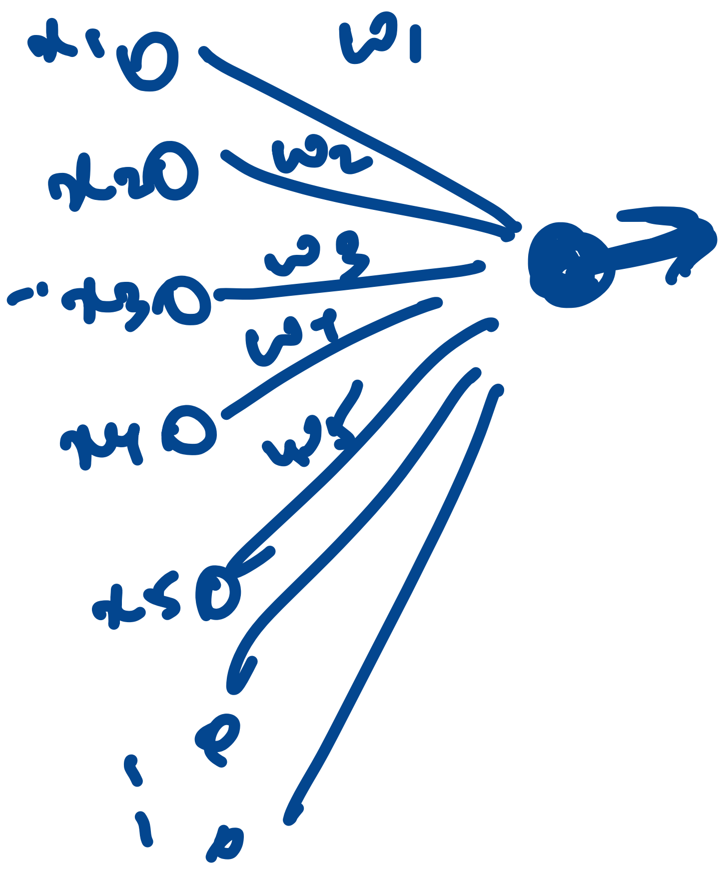
Algo ⇒ Cost function

loss function

Error function

MSE

$z = (w_1 x_1 + w_2 x_2 + w_3 x_3 \dots + b)$



$\hat{y}$ = Activation function (z)

$= \sigma(z)$

$= \frac{1}{1+e^{-z}}$

Activation functions

ReLu

Sigmoid

$\boxed{\text{Tanh}}$ → -1 & 1

$\boxed{\text{Softmax}}$ → Digits → Probability

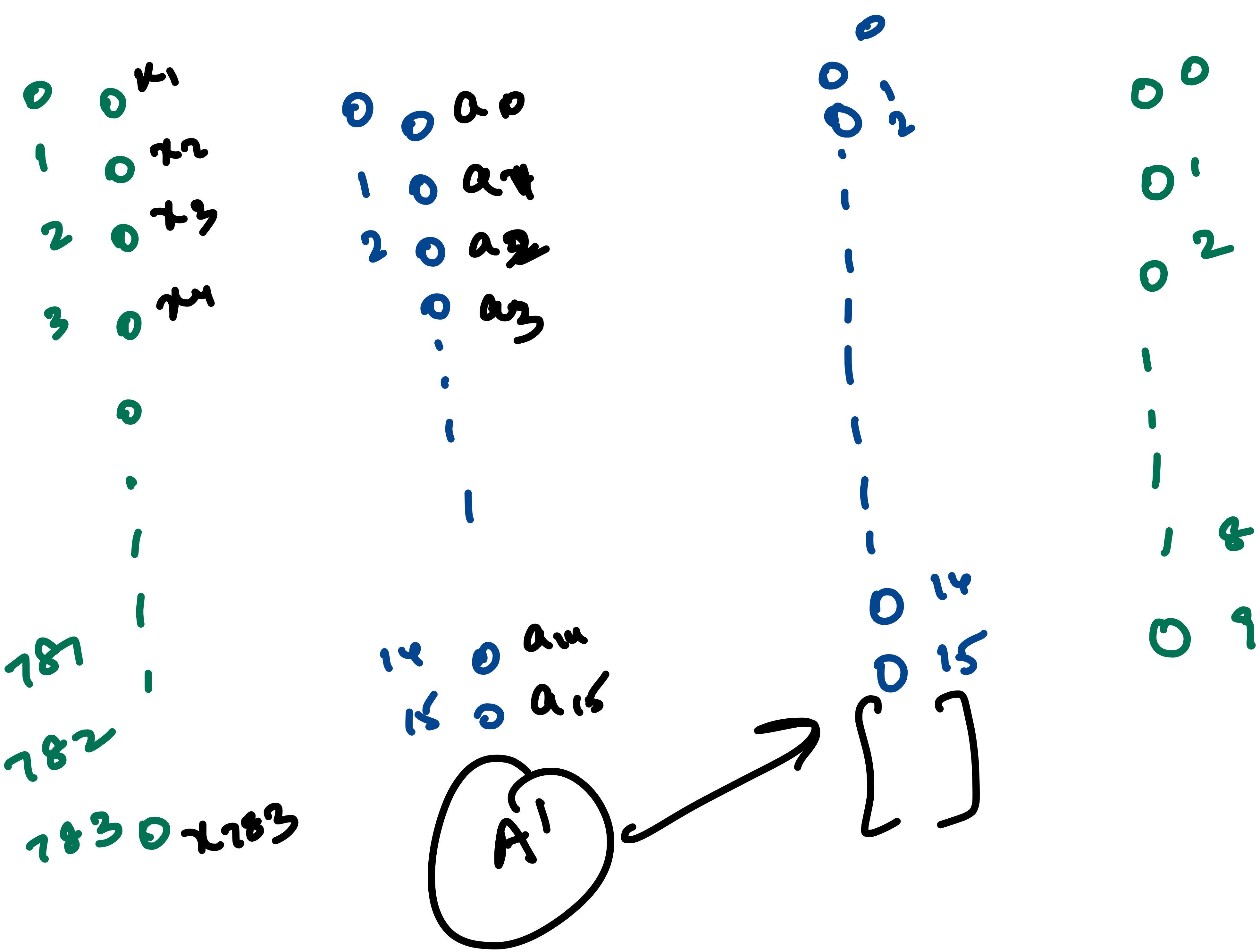
0 - 9 Total probab = 1

Leaky ReLu

$28 \times 28 \Rightarrow 784$ pixels

o/p ⇒ 10 nos
(probability for 10 no.s)

$\boxed{\text{Dense layers}}$ ⇒ All neurons



784 neurons i/p

16 neurons

16 neurons

10 neurons o/p

wts = 784×16 + 16×16 + 16×10

= 12960

biases ⇒ 16 + 16 + 10

= 42

Neuron → Activation value = Activation func (z)

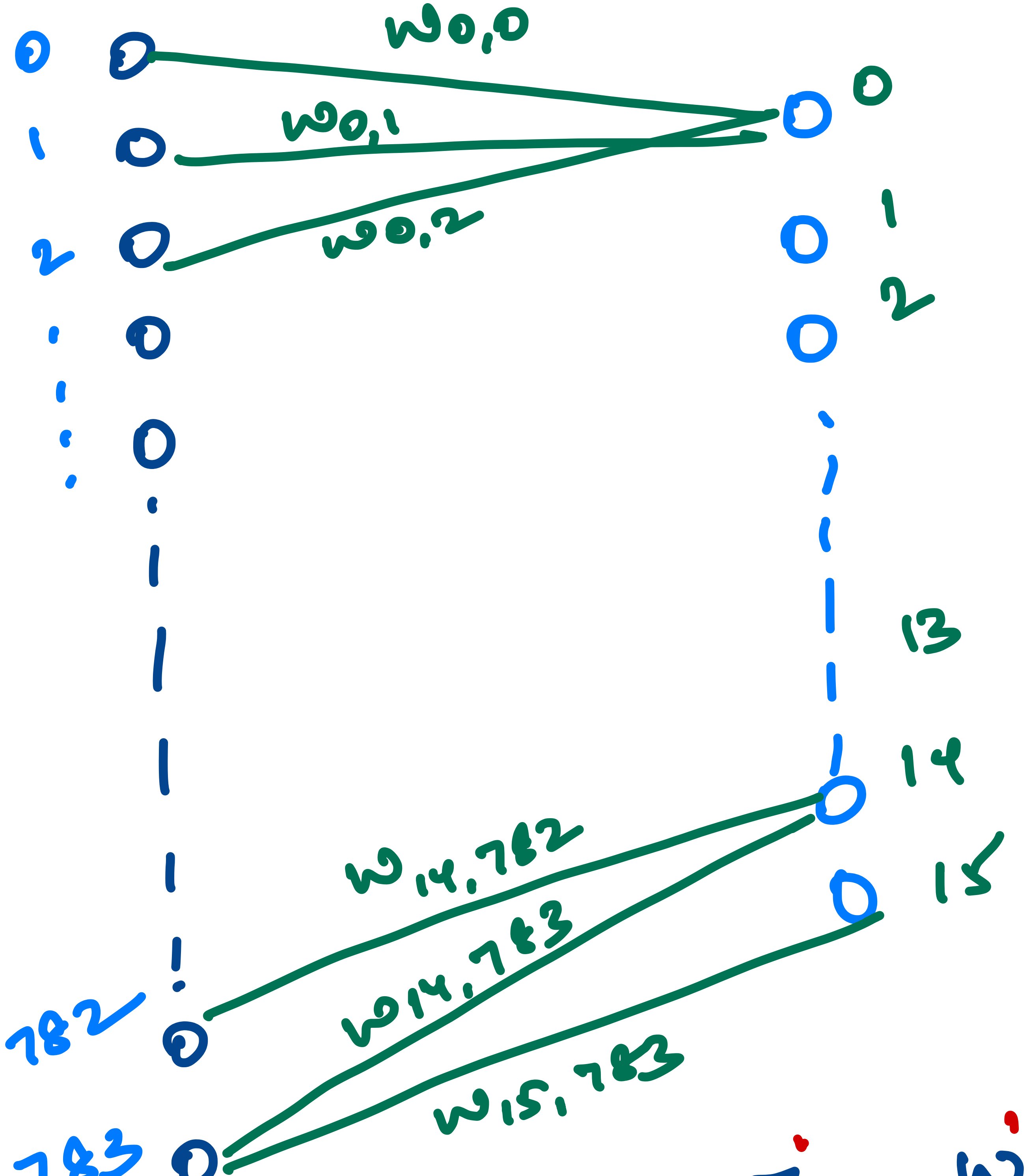
bias

$z = w_1x_1 + w_2x_2 + \dots + b$

neuron

Params ⇒ 12960 + 42

= 13002



Activation vector ⇒ (16 × 1)

Biases vector ⇒ (16 × 1)

$$z = w x + b$$

$$(16 \times 784) \qquad (784 \times 1) \qquad \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_{15} \end{bmatrix}$$

$$Z = \underbrace{W}_{\substack{\text{weight} \\ \text{vector} \\ (16*784)}} \underbrace{x}_{(784*1)} + \underbrace{b}_{\substack{\text{bias vector} \\ (16*1)}}$$

$$\underbrace{(16*784)*(784*1)}_{(16*1)} + (16*1)$$

$$Z \Rightarrow (16*1)$$

$$a_0 = (w_{0,0}x_1 + w_{0,1}x_2 + w_{0,2}x_3 \dots w_{0,783}x_{783} + b)\sigma$$

$$a_1 = (w_{1,0}x_1 + w_{1,1}x_2 - - \cdot - + b)\sigma$$

$$\begin{bmatrix} w_{0,0} & w_{0,1} & w_{0,2} & \cdots & w_{0,783} \\ w_{1,0} & w_{1,1} & w_{2,1} & \cdots & w_{1,783} \\ \vdots & & & & \\ w_{15,0} & w_{15,1} & w_{15,2} & \cdots & w_{15,783} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{783} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_{1k} \end{bmatrix} \quad \sigma \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{15} \end{bmatrix}$$

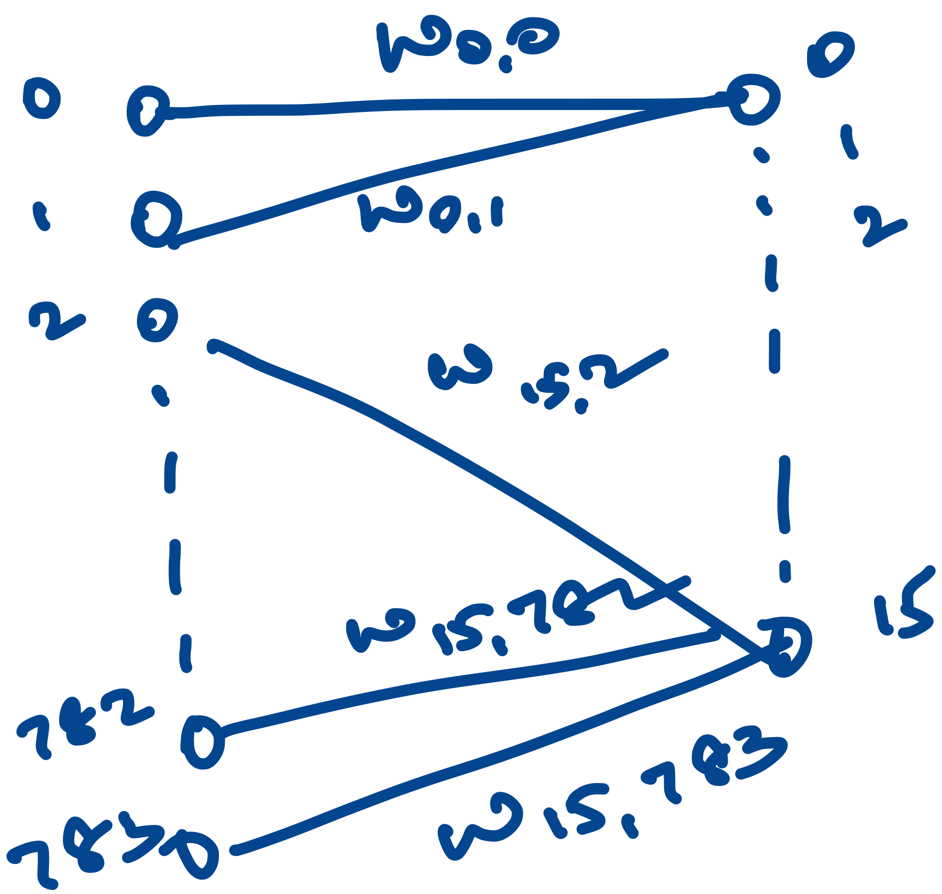
$$\Downarrow$$

$$W$$

$$\left(16 * 1 + 16 * 1 \right)$$

$$Z^1 = 16 * 1$$

$$A^1 = \text{Activation}(z^1)$$



16 neurons

$W = 16 * 16$

16 neurons

$b = 16 * 1$

$$Z^2 = W^2 A^1 + b^2$$

$$\underbrace{(16*16)\,(16*1)}_{16*1} + 16*1$$

($W^2 \to (16*16)$, $A^1 \to (16*1)$, $b^2 \to 16*1$)

$Z^2 \Rightarrow 16*1$

$A^2 =$ Activation (Z^2)
fun

$$Z^1 = W^1 x + b^1$$

Z^1: 16×1

W^1: (16×784)

x: i/p (784×1)

b^1: (16×1)

$$A^1 = \text{Act Function}(z)$$

A^1: 16×1

$$Z^2 = W^2 A^1 + b^2$$

Z^2: 16×1

W^2: (16×16)

A^1: (16×1)

b^2: 16×1

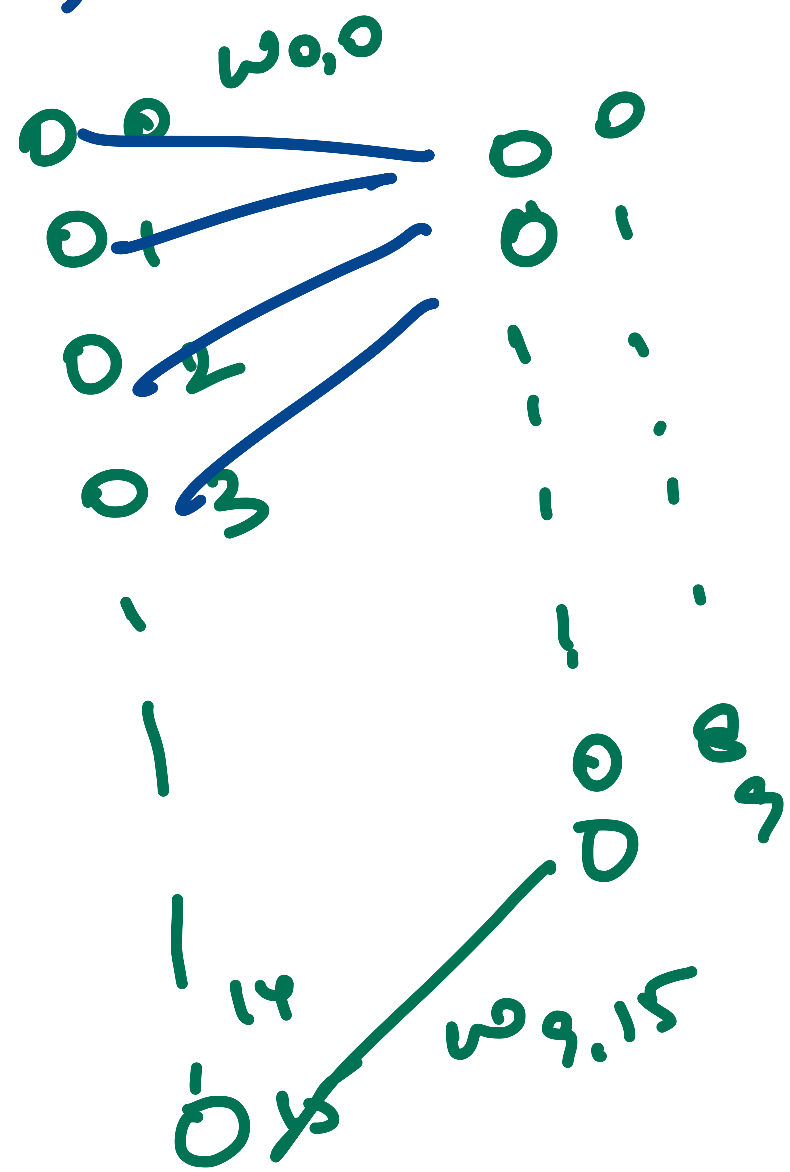
$$A^2 = \text{Activation Func}(Z^2)$$

A^2: 16×1

16 neurons

10 neurons

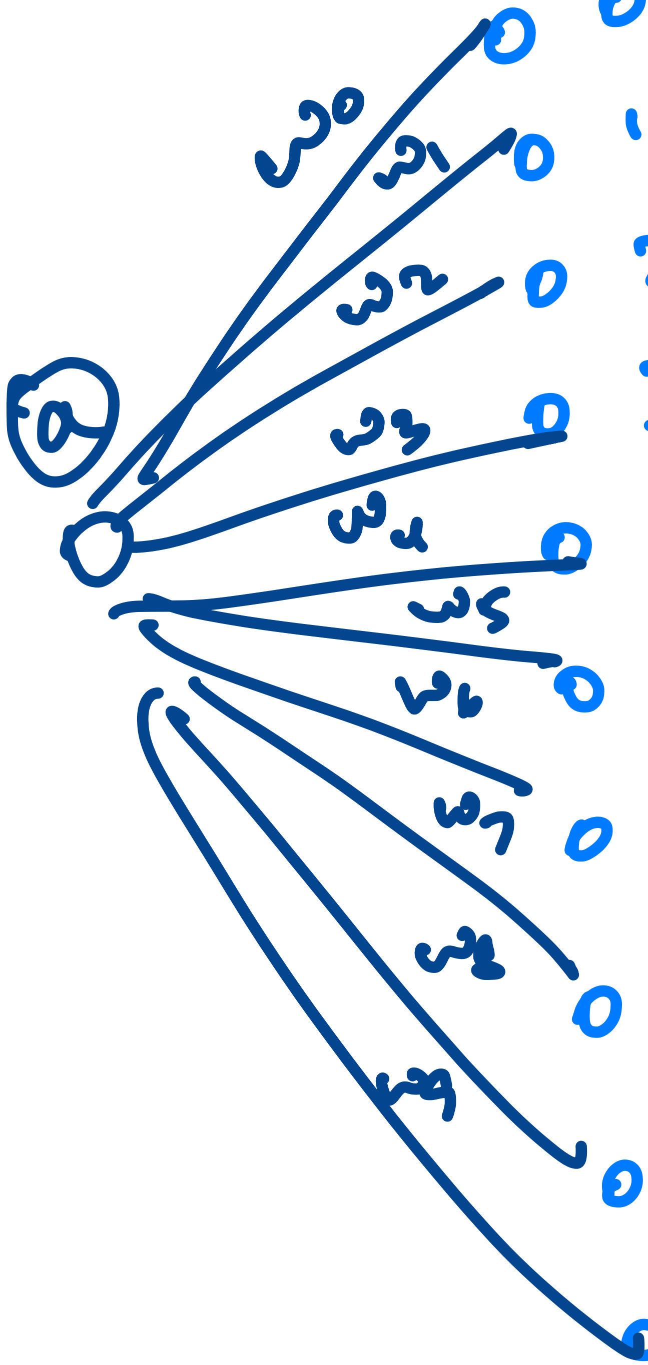
(10*1)



$$Z^3 = \underbrace{W^3}_{(10*16)} \underbrace{A^2}_{(16*1)} + \underbrace{b^3}_{(10*1)}$$

$$o/p = \text{softmax}(Z^3)$$

$$\begin{bmatrix} w_{0,0} & w_{0,1} & \cdots & w_{0,15} \\ w_{1,0} & w_{1,1} & \cdots & w_{1,15} \\ \vdots & & & \\ w_{9,0} & w_{9,1} & \cdots & w_{9,15} \end{bmatrix}$$



0 → 0.1 = $aw_1 + b_1$ (w0 written above)

1 → 0.2 $aw_2 + b_2$

2 → 0.3

3 → 0.1 ⇒ close to 1

4 → 0.8 ⇒ close to 0

5 → 0.7 ⇒ close 0

6 → 0.9 ⇒ close 0

7 → 0.1

8 → 0.6

9 → 0.1

Total = 1

$\sigma(aw_0 + b)$

$y_{true} = 3$

$W = W \pm$ ___ optimization Algo

I have taken Random values but in reality total = 1

100K training Data

Test Data

Validation Data

$\Downarrow$

Batches

100 Data

1K Batches $\rightarrow$ 100 Data Points

100K

updating weights, Bias

After every Batch

100 data

Ytrue

3 5 2 6 7 - - - - - ' 3 Avg

$w_{0,0}$ +0.01 +0.02 -0.03 - - - - - - .

$w_{0,1}$

$0_{1,1}$

(1) Fwd Pass

Loss

(2) $\frac{\partial L}{\partial w}$ $\frac{\partial L}{\partial b}$

Batch

avg change

w, b

⇓

Batch

(3) optimization for batch

Image → 28 × 28 pixel

1K × 1K

I/p neurons → 10^6 neurons

1 million neuron

100 neuron in 1st hidden

wts → 100 million

CNN

⇓

Images

$\boxed{\text{RNN}}$ → next week

0, 255
black
white

0 0 0 0 0 0
0 255 255 255 255 0
0 255 255 255 255 0
0 255 255 255 255 0
0 0 0 255 0 0
0 0 255 255 0 0
0 255 255 0 0 0
0 0 0 0 0 0

1 0 1
1 0 1
1 0 1

0 - 1
0 - 1
0 - 1

0 - 0
255 - 0
255 - 0

0 + 1
255 + 1
255 + 1

Image $\Rightarrow \begin{bmatrix} 0 & 255 & 255 \\ 0 & 255 & 255 \\ 0 & 255 & 255 \end{bmatrix}$

vertical

Edge Detection

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

flip

$$\begin{array}{ccc} \text{Black} & & \text{White} \end{array}$$

Matrix $\Rightarrow \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$

Ans $\Rightarrow$ +ve white on Right side

Image $\Rightarrow \begin{bmatrix} 255 & 255 & 0 \\ 255 & 255 & 0 \\ 255 & 255 & 0 \end{bmatrix}$

White Black

$-255 - 255 - 255 + 0 + 0$

$\Rightarrow -255 \Rightarrow$ white on left side

Black on Right side

Horizontal Edge Detection

$$\begin{array}{c}\begin{bmatrix} 255 & 255 & 255 \\ 255 & 255 & 255 \\ 0 & 0 & 0 \end{bmatrix}\end{array} \begin{array}{l} \text{White} \\ \\ \text{Black} \end{array}$$

Sum(Dot Product)

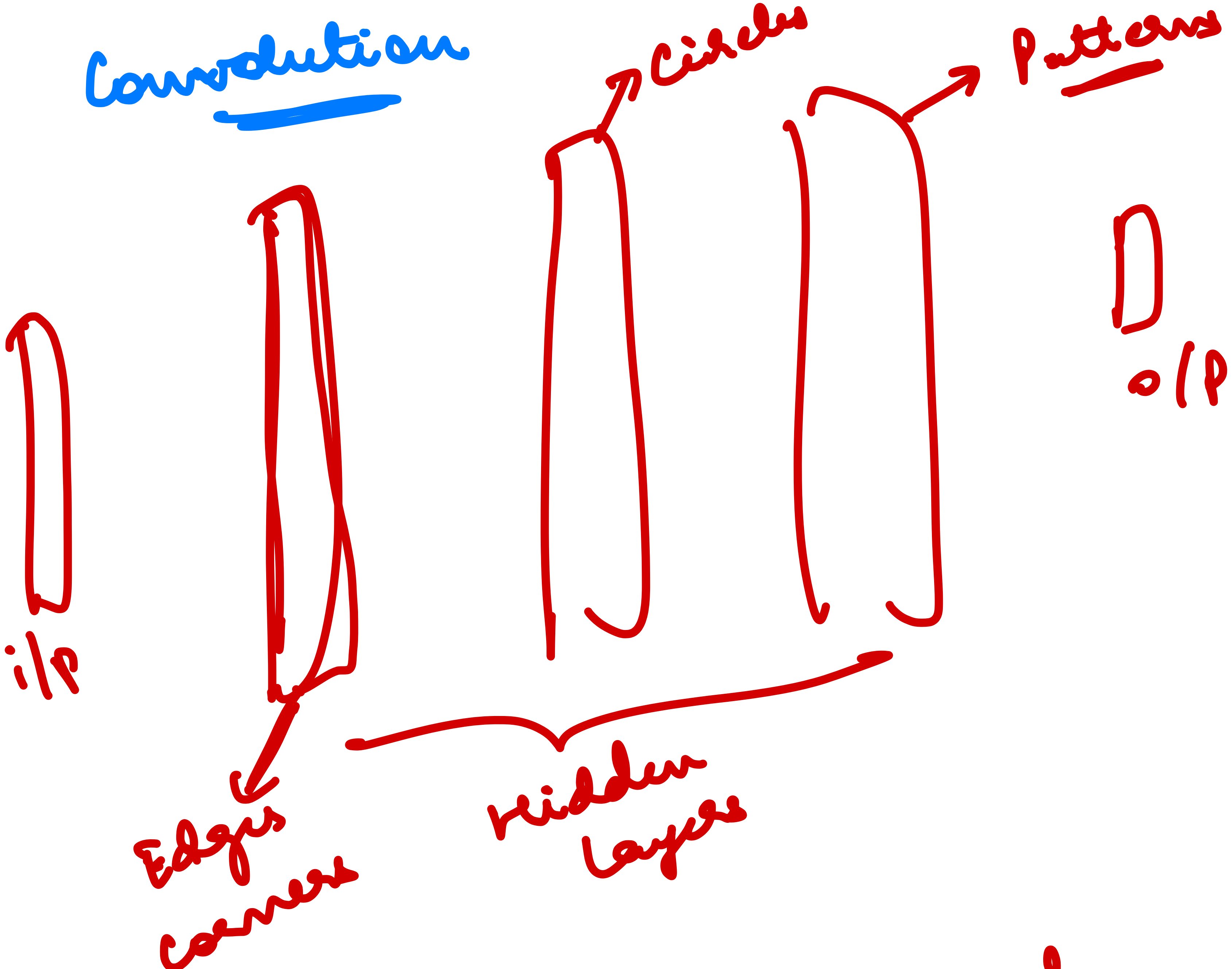
Matrix $\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$

$-255 \ -255 \ -255 \Rightarrow -ve$

$$\begin{bmatrix} 0 & 0 & 0 \\ 255 & 255 & 255 \\ 255 & 255 & 255 \end{bmatrix} \begin{array}{l} \text{Black} \\ \text{White} \\ \\ \end{array}$$

$0 + 0 + 0 + 0 + 0 + 0 + 255 + 255 + 255 \Rightarrow +ve$

Convolution



CNN ⇒ Convolutional Neural Network

Kernel / Filter

Vertical Edge Kernel $= \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$

Horizontal Edge Kernel $= \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$

Diagonal $= \begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix}$

$\begin{bmatrix} 2 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & -1 & -2 \end{bmatrix}$

$$\begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix} \begin{bmatrix} 255 & 0 & 0 \\ 255 & 255 & 0 \\ 255 & 255 & 255 \end{bmatrix}$$

$0 + 0 + 0 + (-255) + 0 + 0$

$(-2 \cdot 255) - 255 + 0$

$\Rightarrow -ve$

$$\begin{matrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{matrix}$$

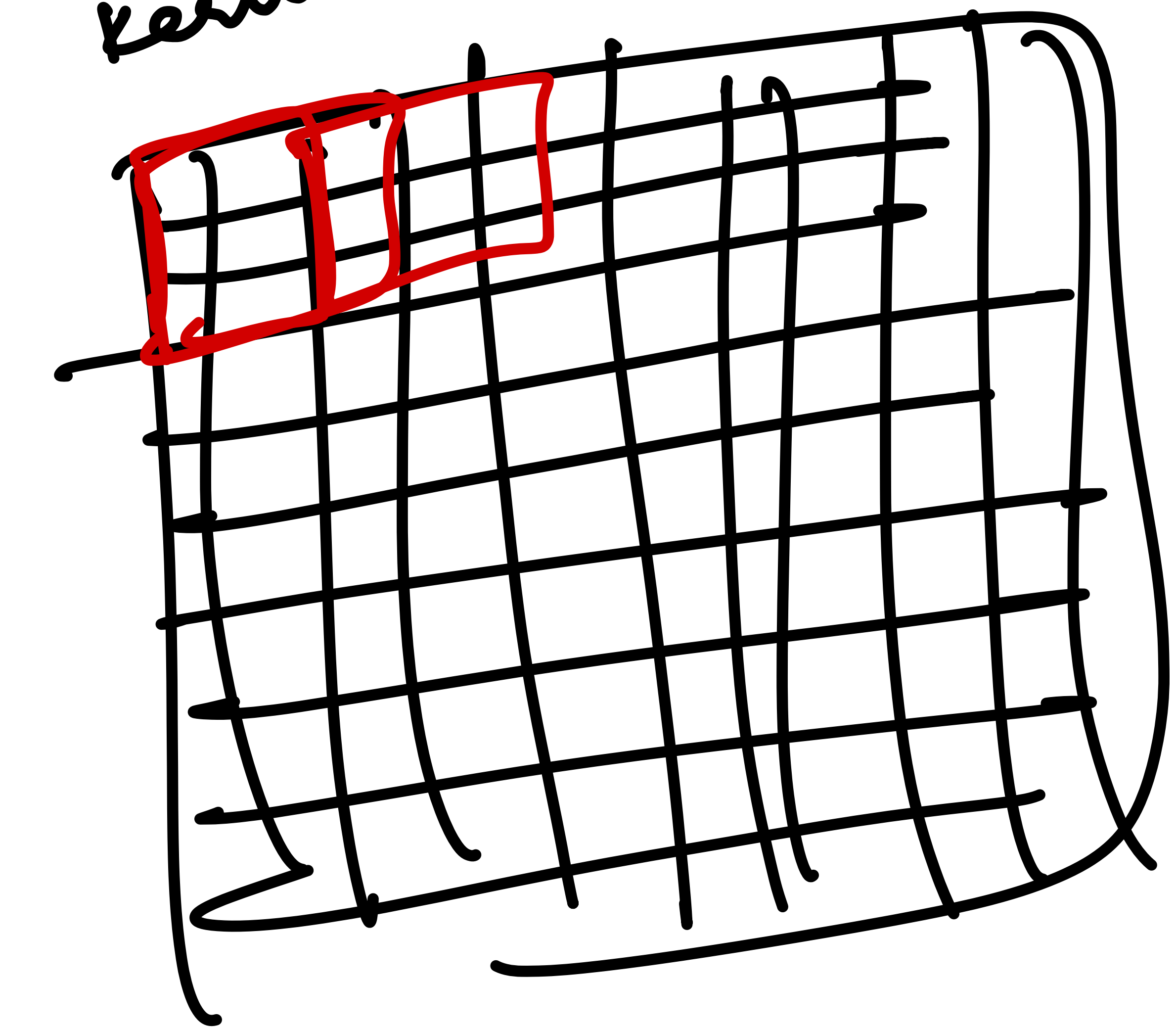
black and white

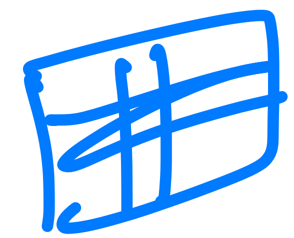
⇓

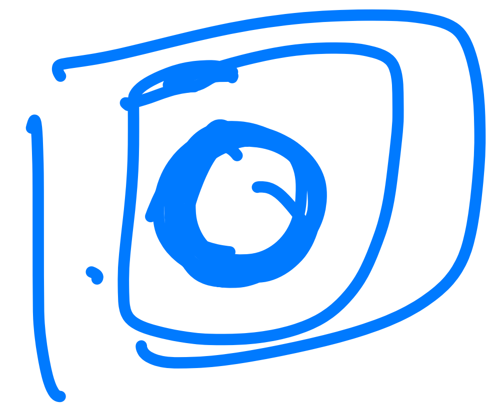
coloured

Image $3*3$ → 999 & 999

Kernel $3*3$







CNN

⇓

Filters

Start → Filters

Bias

ANN → weights

+

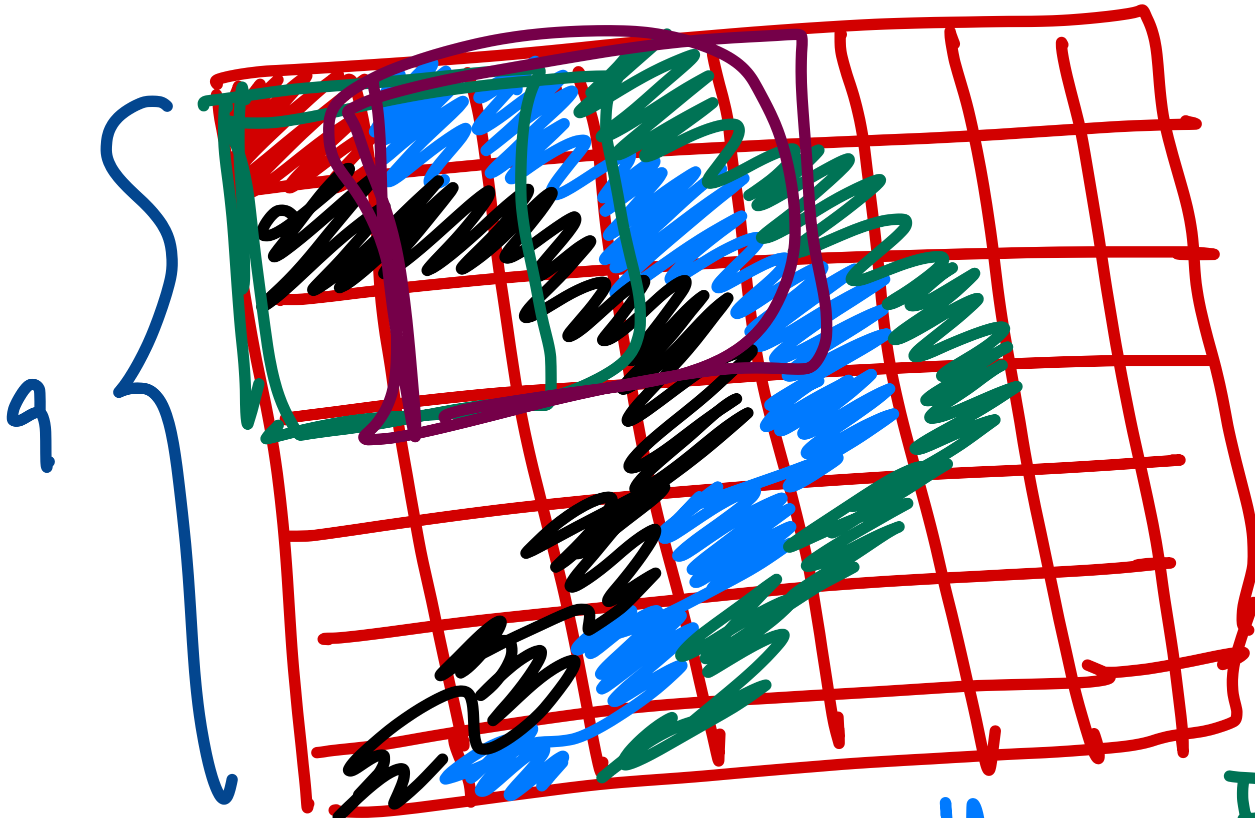
Bias

hyperparameter → Size of kernel/ filter

Stride → vertical

horizontal

Blurring →



Downsampling

$\Downarrow$

upsample

$\Downarrow$

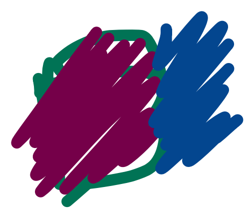
3×3

Dimensionality

Reduction

$\Downarrow$

Blurring



Autoencoder

MNIST Data Set

7 ⇒ 28 × 28

⇓

encode

smaller dimension → latent space

↓

decode

Dense Layer ⇒ Fully Connected Layer

⇑

Flatten
1D array

Convolutional Layer

Batch × W × H × channels

⇓ ⇓ ⇓

28 28

channel ⇒ no. of colors per pixel

grayscale ⇒ 1 (intensity of black & white)

(28, 28, 3)

RGB ⇒ 3 channels

RGBA ⇒ 4 channels

RGB with alpha

⇓

transparency

((32), 256, 256, 3)

Batch

RGB image